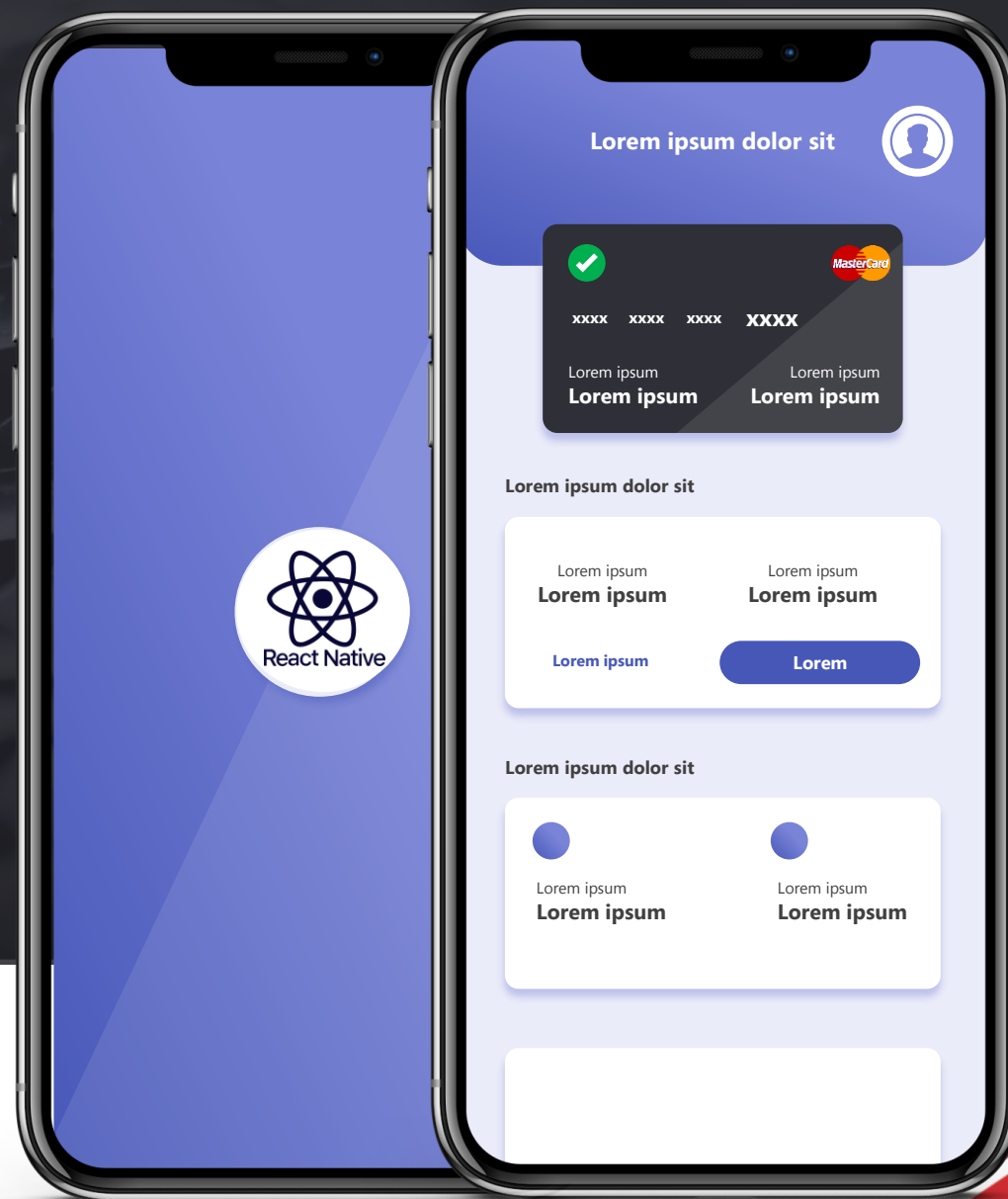




Programação para DISPOSITIVOS MÓVEIS

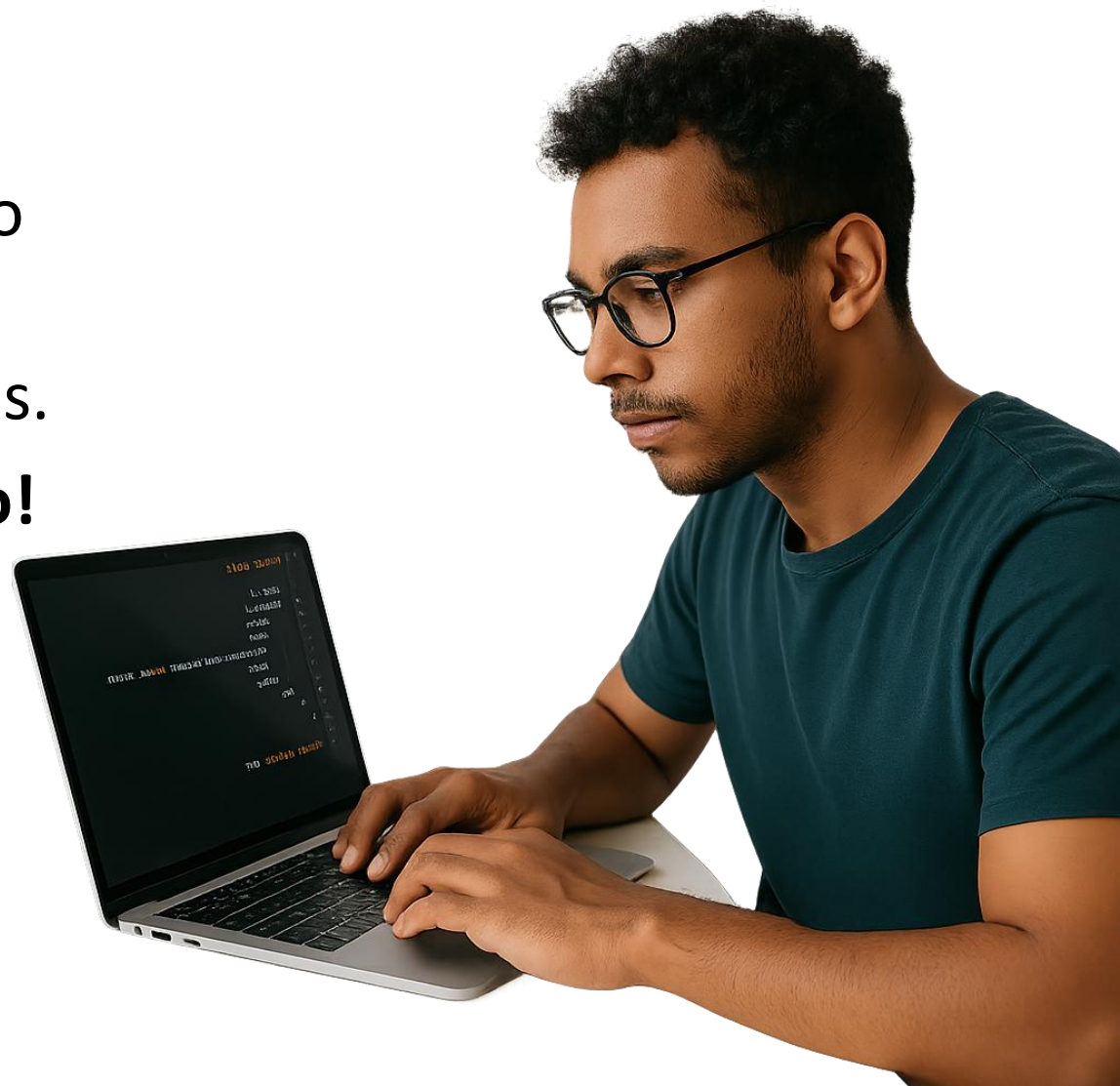
Aula 07: Navegação

Prof. José Alberto S. Torres



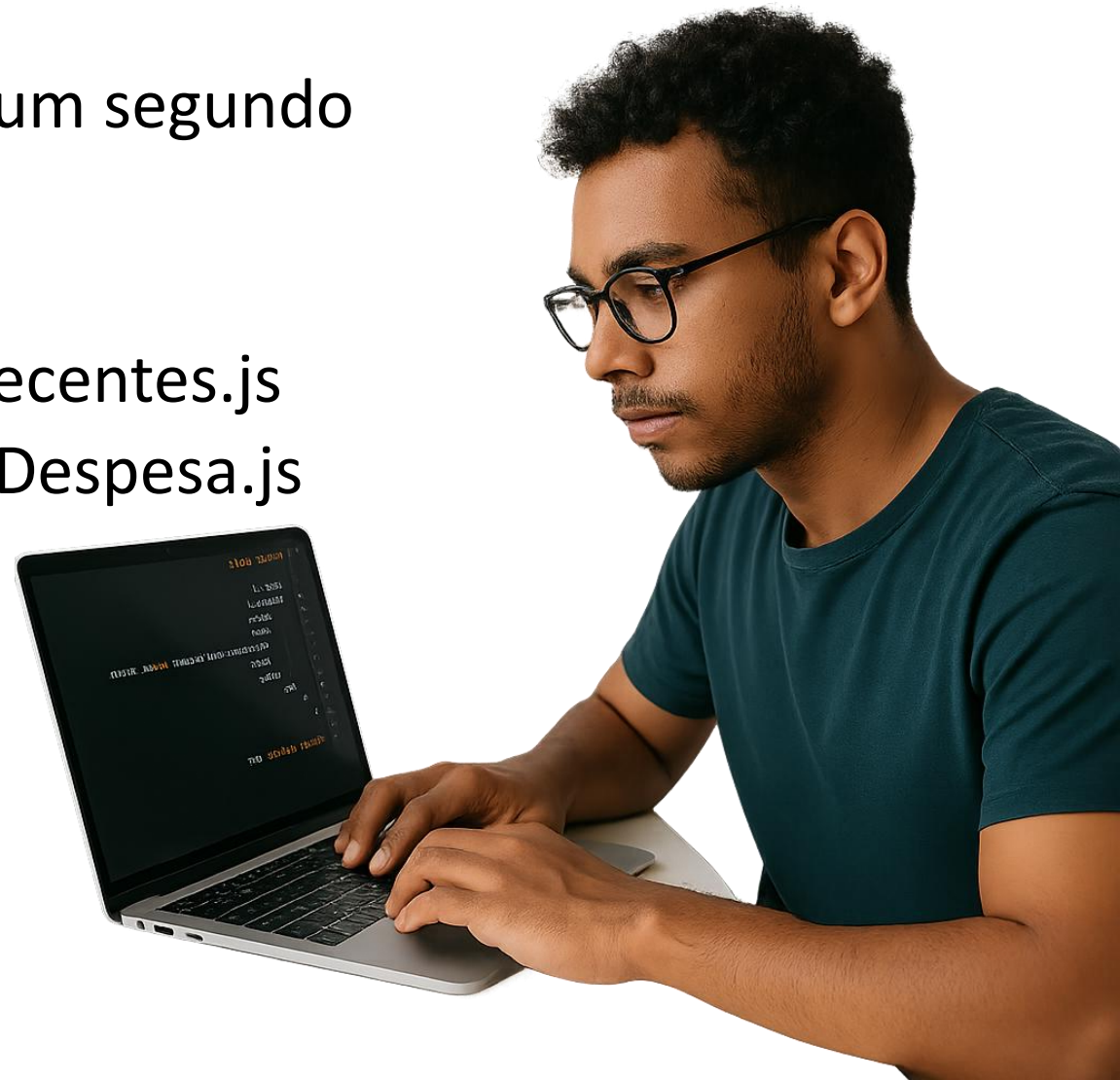
Meu Segundo Projeto

- Oportunidade de **desenvolver** e **aperfeiçoar** o conteúdo aprendido com o primeiro projeto.
- Oportunidade de desenvolver novas skills.
- Para isso, vamos **iniciar um novo projeto!**
- **Crie um novo diretório e inicie um novo projeto React Native via expo!**



Meu Segundo Projeto

- Dentro do diretório do seu projeto, **crie** um segundo diretório chamado **screens**.
- **Dentro de screens:**
 - Criar um arquivo chamado DespesaRecentes.js
 - Criar um arquivo chamado GerenciarDespesa.js
 - Criar um arquivo chamado TodasDespesas.js



Meu Segundo Projeto

```
import {Text} from 'react-native'

function DespesaRecente(){

  return (
    <Text>Despesa Recente</Text>
  )
}

export default DespesaRecente
```

```
> .expo
> assets
> node_modules
▼ screens
  DespesasRecentes.js
  GerenciarDespesa.js
  TodasDespesas.js
  .gitignore
  App.js
  app.json
  index.js
  package-lock.json
  package.json
```



Atividade

- A atividade consiste em:
 - Criar um arquivo chamado TodasDespesas.js
 - Criar um arquivo chamado GerenciarDespesa.js

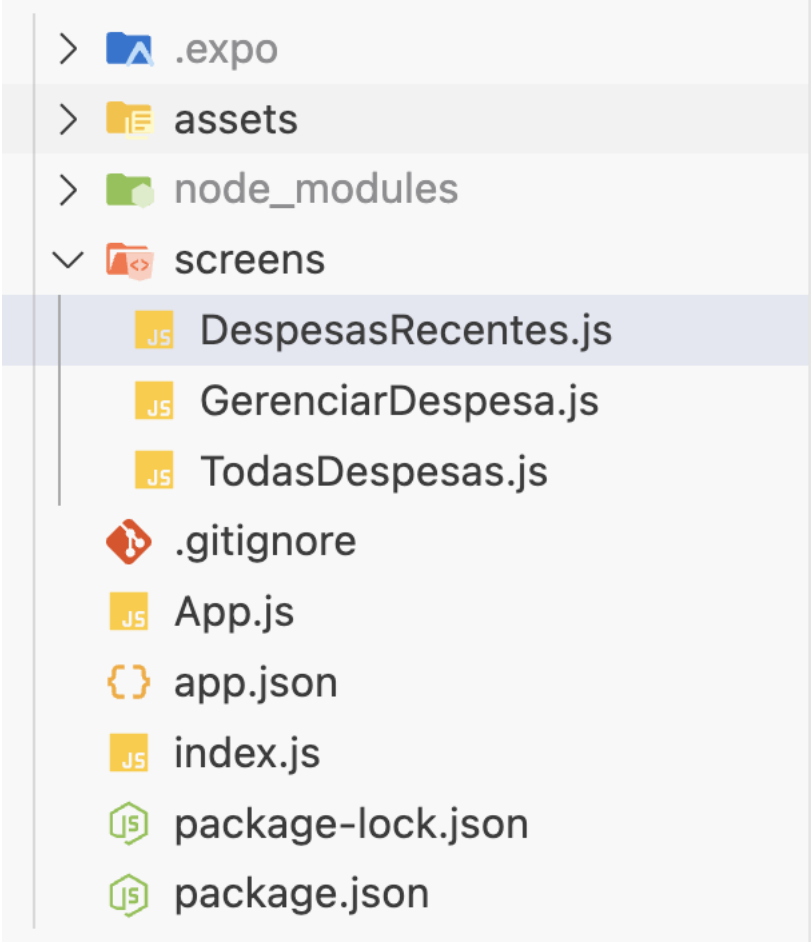
Meu Segundo Projeto

```
import {Text} from 'react-native'

function GerenciarDespesa(){

  return (
    <Text>GerenciarDespesa</Text>
  )
}

export default GerenciarDespesa
```



```
> .expo
> assets
> node_modules
▼ screens
  DespesasRecentes.js
  GerenciarDespesa.js
  TodasDespesas.js
  .gitignore
  App.js
  app.json
  index.js
  package-lock.json
  package.json
```

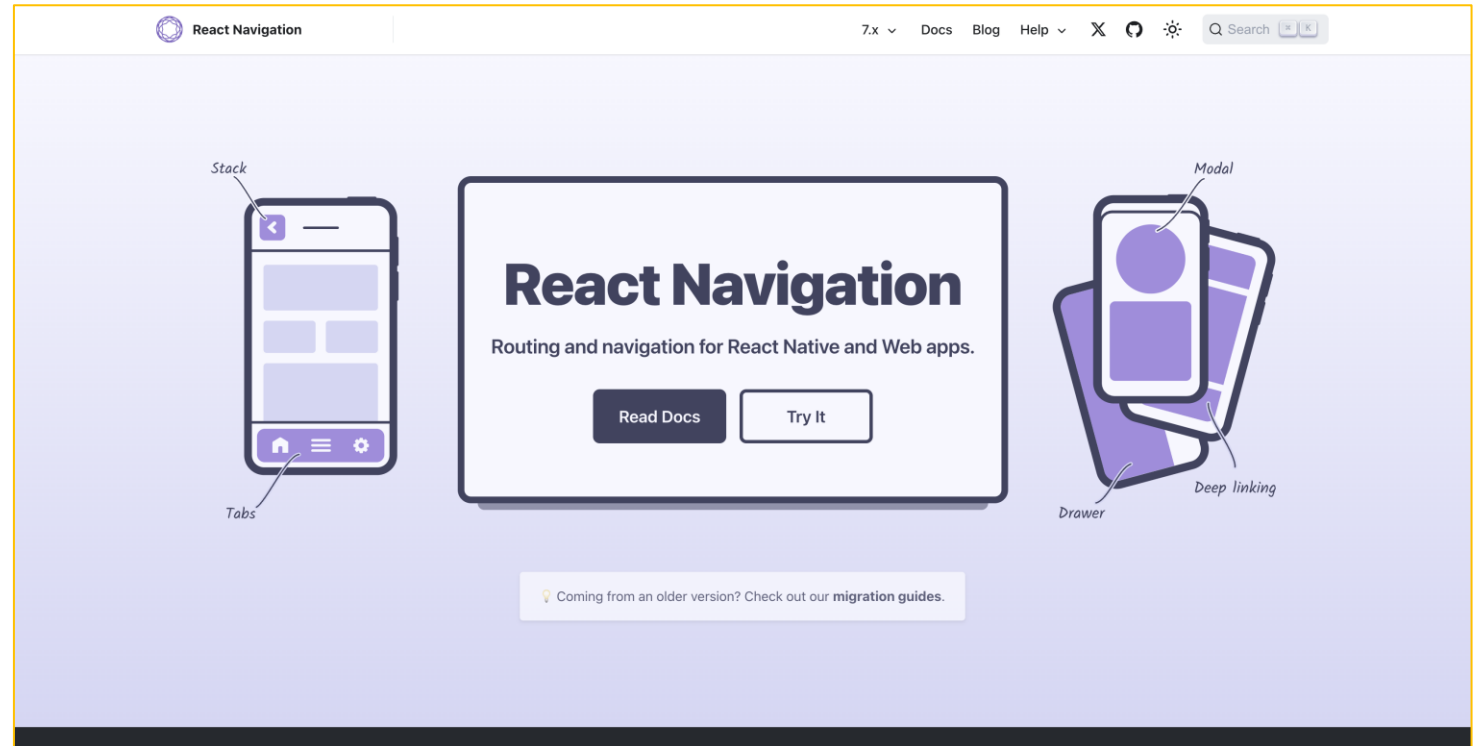
Meu Segundo Projeto

```
aula_09 > screens > JS TodasDespesas.js > [🔗] default
1  import {Text} from 'react-native'
2
3  function TodasDespesas(){
4
5      return (
6          <Text>TodasDespesas</Text>
7      )
8
9  }
10
11  export default TodasDespesas
```

```
> .expo
> assets
> node_modules
▼ screens
  JS DespesasRecentes.js
  JS GerenciarDespesa.js
  JS TodasDespesas.js
  .gitignore
  JS App.js
  app.json
  JS index.js
  package-lock.json
  package.json
```

Navegação

- No **React Native**, a navegação entre telas (ou “views”) é feita usando bibliotecas específicas, sendo a principal delas o **React Navigation**.



<https://reactnavigation.org>



React Navigation

- É uma **biblioteca de navegação** que permite a criação de **fluxos de telas** em apps React Native — como ir de uma tela pra outra, voltar, usar abas, menus laterais, etc.
- Ele simula o comportamento de **navegação em apps nativos**.



Navigators

- São componentes fornecidos pela biblioteca React Navigation que definem a estrutura e o comportamento da navegação dentro do seu aplicativo.
- Eles permitem que você organize e gerencie as diferentes telas (ou “screens”) da sua aplicação, controlando como os usuários se movem entre elas.



Navigators

- Os navigators são responsáveis por:
 - **Definir a estrutura de navegação:** Eles organizam as telas do aplicativo em uma hierarquia lógica.
 - **Gerenciar transições entre telas:** Controlam como as telas aparecem e desaparecem, incluindo animações e gestos.
 - **Manter o estado de navegação:** Acompanham o histórico de navegação, permitindo ações como voltar para a tela anterior.
 - **Fornecer elementos de interface:** Como cabeçalhos, abas ou menus laterais, dependendo do tipo de navigator utilizado.



Tipos comuns de Navigators

- Stack
- Native Stack
- Bottom Tabs
- Drawer
- Material Top Tabs

Exemplo:

```
import { createStackNavigator } from '@react-navigation/stack';

const Stack = createStackNavigator();

function MyStack() {
  return (
    <Stack.Navigator>
      <Stack.Screen name="Home" component={HomeScreen} />
      <Stack.Screen name="Profile" component={ProfileScreen} />
    </Stack.Navigator>
  );
}
```



Navigation Container

- Atua como o **contêiner principal que gerencia o estado de navegação da sua aplicação** e integra os navegadores (navigators) ao ambiente do aplicativo.
- **Sem o NavigationContainer, os navegadores não funcionam corretamente**, pois é ele que fornece o contexto necessário para a navegação.

Navigation Container

- Exemplo:

```
import { createStackNavigator } from '@react-navigation/stack';
import { NavigationContainer } from '@react-navigation/native';

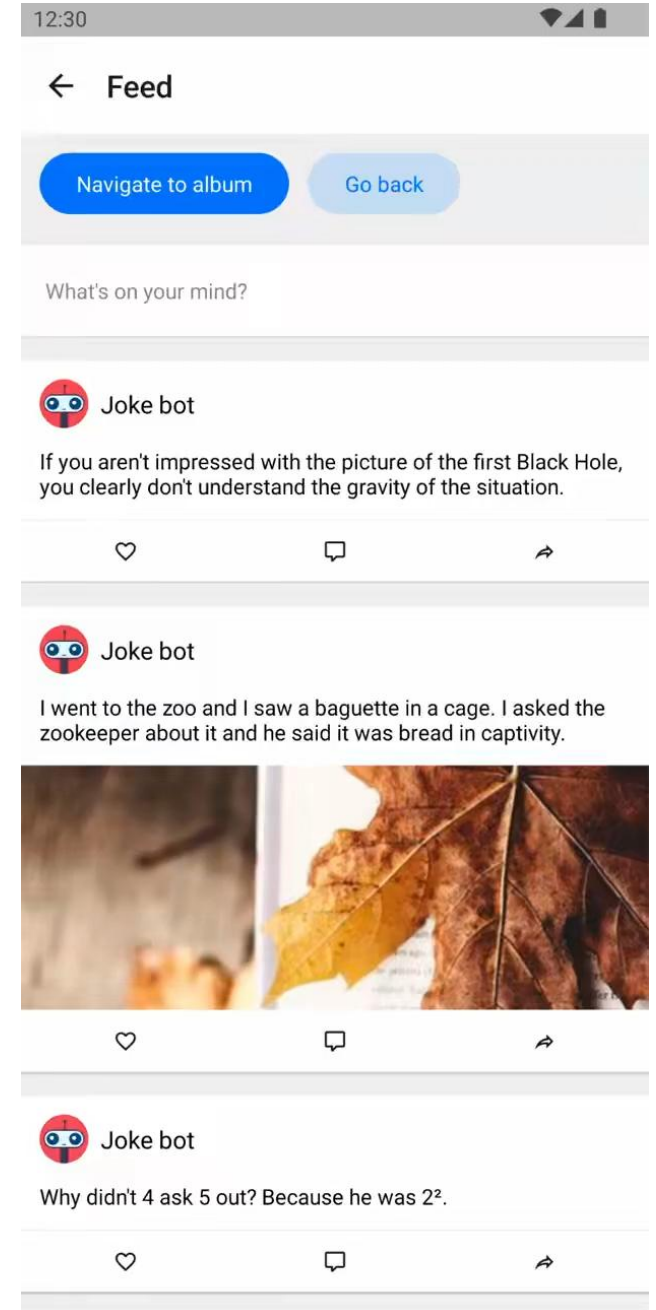
const Stack = createStackNavigator();
function MyStack() {
  return (
    <NavigationContainer>

      <Stack.Navigator>
      <Stack.Screen name="Home" component={HomeScreen} />
      <Stack.Screen name="Profile" component={ProfileScreen} />
    </Stack.Navigator>

    <NavigationContainer>
  );
}
```

Stack

- Navegação **em pilha**, como páginas sendo empilhadas.
 - Cada tela é empilhada sobre a anterior.
 - Pode voltar com `goBack()`.
 - Comportamento parecido com navegadores ou apps de chat.





Stack

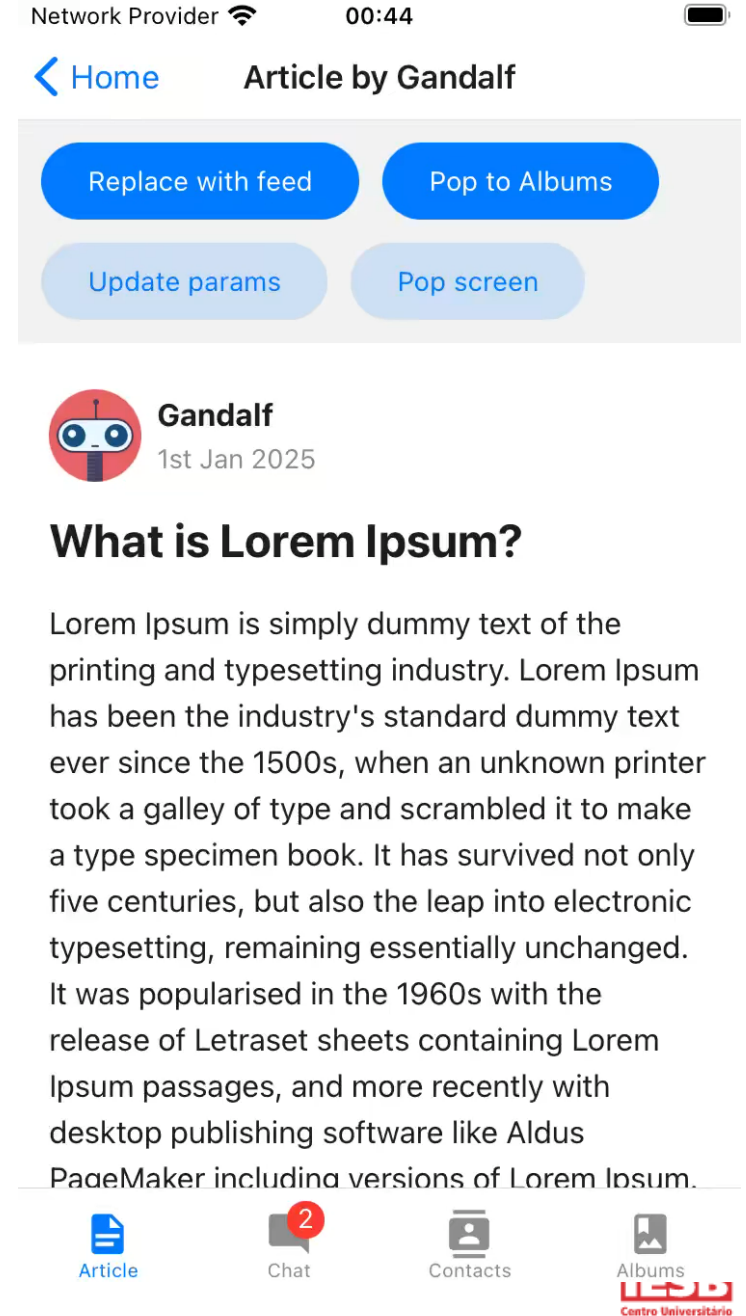
- Para usar este navegador, certifique-se de ter instalado:
 - `npm install @react-navigation/native`
 - `npm install @react-navigation/stack`
 - `npx expo install react-native-gesture-handler`

```
import { createStackNavigator } from '@react-navigation/stack';

const Stack = createStackNavigator();
function MyStack() {
  return (
    <Stack.Navigator>
    <Stack.Screen name="Home" component={HomeScreen} />
    <Stack.Screen name="Profile" component={ProfileScreen} />
    </Stack.Navigator>
  );
}
```


Bottom Tabs

- Abas na parte inferior da tela — estilo Instagram, YouTube, etc.
- Fácil de navegar entre seções principais.
- Ícones e labels personalizáveis.



Bottom Tabs

- Para usar este navegador, certifique-se de ter instalado:
 - `npm install @react-navigation/native`
 - `npm install @react-navigation/bottom-tabs`

```
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';

const Tab = createBottomTabNavigator();

function MyTabs() {
  return (
    <Tab.Navigator>
    <Tab.Screen name="Home" component={HomeScreen} />
    <Tab.Screen name="Profile" component={ProfileScreen} />
    </Tab.Navigator>
  );
}
```



Atividade **Prática** 2

- No seu arquivo App.js, instancie:
 - Um Boton Tab Navigator em uma constante de nome Tab.

```
import { StatusBar } from 'expo-status-bar';  
import { StyleSheet, Text, View } from 'react-native';  
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';  
  
export default function App() {  
  
  const Tab = createBottomTabNavigator();
```

Atividade Prática 2

- No seu arquivo App.js, crie uma função de nome BottonTabScreen que retorna <Tab.Navigator></Tab.Navigator>

```
import { StatusBar } from 'expo-status-bar';
import { Text, View } from 'react-native';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';

export default function App() {

  const Tab = createBottomTabNavigator();

  function BottonTabScreen(){
    return(
      <Tab.Navigator>

      </Tab.Navigator>
    )
  }
}
```



Atividade **Prática** 2

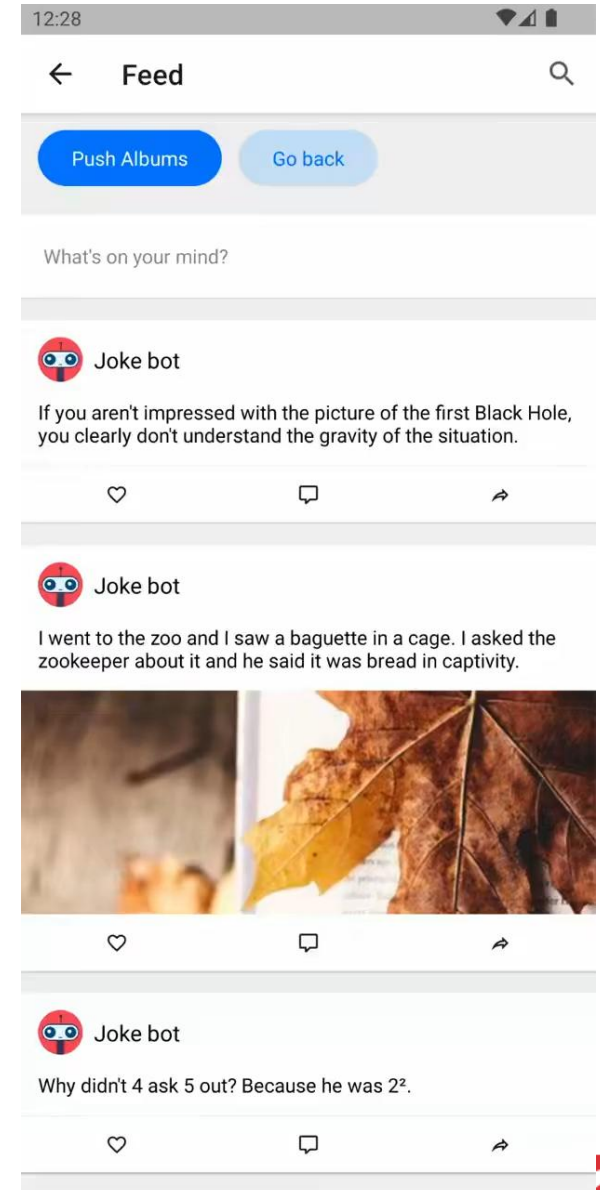
- Dentro de Tab.Navigator:
 - Adicione uma Tab.Screen com “name” igual a “DespesaRecentes” e “componente” igual a DespesaRecentes.
 - Adicione uma Tab.Screen com “name” igual a “TodasDespesas” e “componente” igual a TodasDespesas.
- Não esqueça de importar os componentes!

Atividade Prática 2

```
a_08 > JS App.js > App
1  import { StyleSheet, Text, View } from 'react-native';
2  import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
3  import { createNativeStackNavigator } from '@react-navigation/native-stack';
4  import { NavigationContainer } from '@react-navigation/native';
5  import GerenciarDespesa from './screens/GerenciarDespesa';
6  import DespesaRecentes from './screens/DespesaRecentes';
7  import TodasDespesas from './screens/TodasDespesas';
8
9  export default function App() {
10
11    const Tab = createBottomTabNavigator();
12
13    function BottomTabScreen(){
14      return(
15        <Tab.Navigator>
16          <Tab.Screen name="DespesaRecentes" component={DespesaRecentes} />
17          <Tab.Screen name="TodasDespesas" component={TodasDespesas} />
18        </Tab.Navigator>
19      )
20    }
21  }
```

Native Stack

- Igual ao Stack, mas usa **navegação nativa de verdade**, com animações mais suaves e desempenho melhor.
- Cada tela é empilhada sobre a anterior.
- Requer menos memória e é mais rápido.



Native Stack

- Para usar este navegador, certifique-se de ter instalado:
 - `npm install @react-navigation/native`
 - `npm install @react-navigation/native-stack`

```
import {createNativeStackNavigator} from '@react-navigation/native-stack';

const Stack = createNativeStackNavigator ();
function MyStack() {
  return (
    <Stack.Navigator>
    <Stack.Screen name="Home" component={HomeScreen} />
    <Stack.Screen name="Profile" component={ProfileScreen} />
    </Stack.Navigator>
  ); };
```


Atividade **Prática** 2

- No seu arquivo App.js, instancie:
 - Um Native Stack Navigator em uma constante de nome Stack.
 - Dentro do método return da função App, substitua o código atual por uma estrutura do tipo:
 - `<NavigationContainer>`
 - `<Stack.Navigator></Stack.Navigator>`:
 - `</NavigationContainer>`
 - Não se esqueça de importar as bibliotecas.

Atividade Prática 2

```
import { StyleSheet, Text, View } from 'react-native';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import { NavigationContainer } from '@react-navigation/native';
import DespesaRecentes from './screens/DespesaRecentes';
import TodasDespesas from './screens/TodasDespesas';

export default function App() {

  const Tab = createBottomTabNavigator();

  function BottomTabScreen(){
    return(
      <Tab.Navigator>
        <Tab.Screen name="DespesaRecentes" component={DespesaRecentes} />
        <Tab.Screen name="TodasDespesas" component={TodasDespesas} />
      </Tab.Navigator>
    )
  }

  const Stack = createNativeStackNavigator();
  return (
    <NavigationContainer>
      <Stack.Navigator>

        </Stack.Navigator>
    </NavigationContainer>
  );
}
```



Atividade **Prática** 2

- Dentro de Stack.Navigator:
 - Crie uma “Stack.Screen” de “name” igual a Despesas e “component” igual a BottonTabScreen
 - Uma “Stack.Screen” de “name” igual a GerenciarDespesa e “component” igual a GerenciarDespesa.

Atividade Prática 2

```
const Stack = createNativeStackNavigator();

return (
  <NavigationContainer>
    <Stack.Navigator>
      <Stack.Screen name="Despesas" component={BottonTabScreen} />
      <Stack.Screen name="GerenciarDespesa" component={GerenciarDespesa} />
    </Stack.Navigator>
  </NavigationContainer>
);
}
```

Atividade Prática 2

```
import { StyleSheet, Text, View } from 'react-native';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import { NavigationContainer } from '@react-navigation/native';
import GerenciarDespesa from './screens/GerenciarDespesa';
import DespesaRecentes from './screens/DespesaRecentes';
import TodasDespesas from './screens/TodasDespesas';

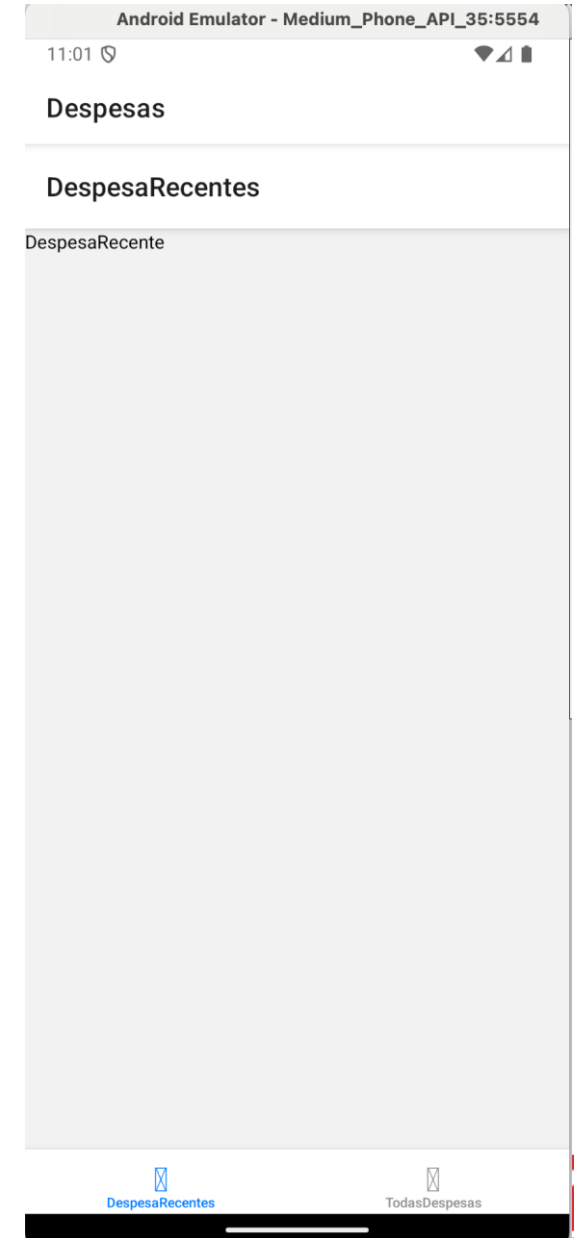
export default function App() {

  const Tab = createBottomTabNavigator();

  function BottonTabScreen(){
    return(
      <Tab.Navigator>
        <Tab.Screen name="DespesaRecentes" component={DespesaRecentes} />
        <Tab.Screen name="TodasDespesas" component={TodasDespesas} />
      </Tab.Navigator>
    )
  }

  const Stack = createNativeStackNavigator();
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen name="Despesas" component={BottonTabScreen} />
        <Stack.Screen name="GerenciarDespesa" component={GerenciarDespesa} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}

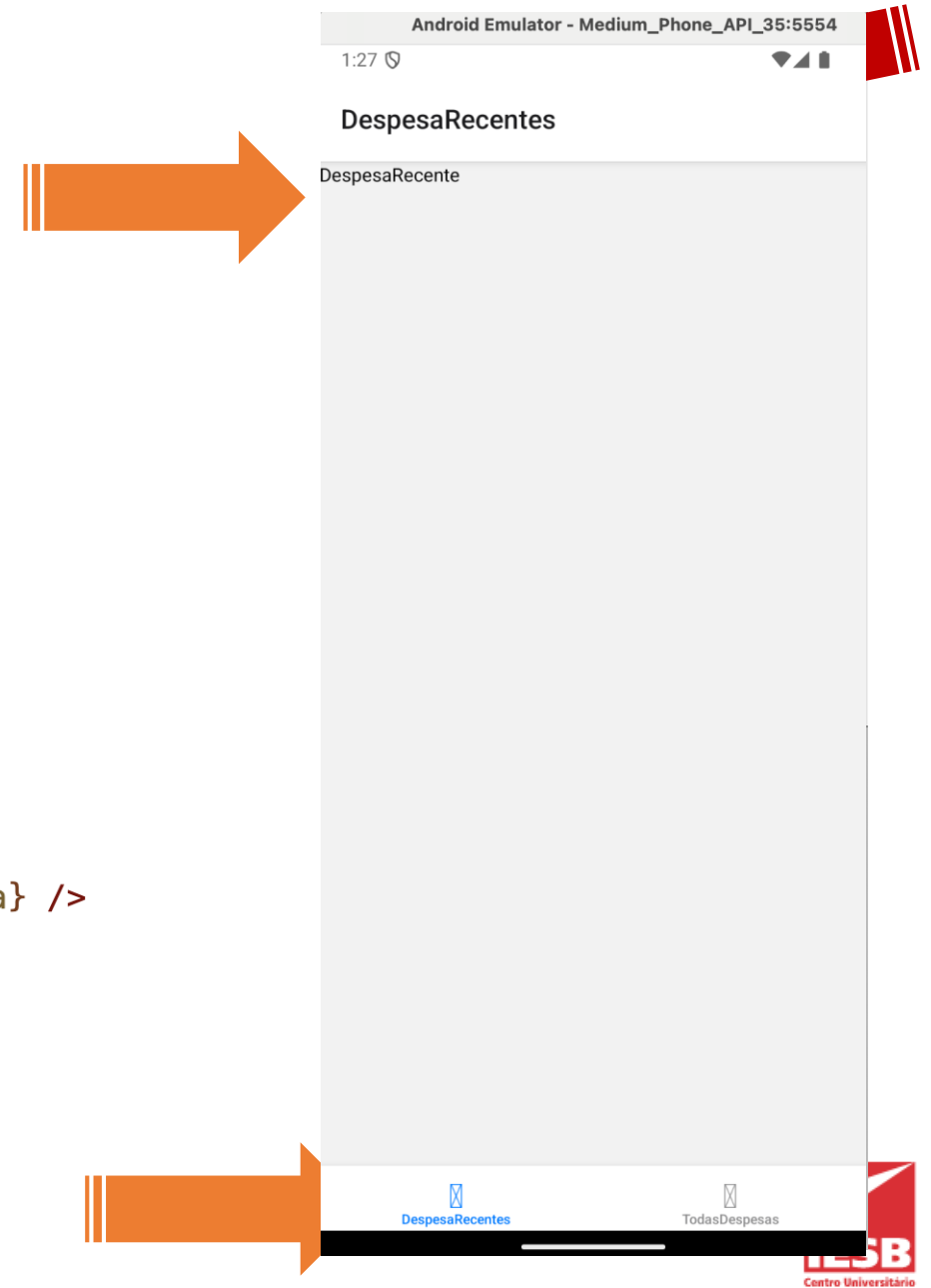
const styles = StyleSheet.create({ container: {
  flex: 1,
  backgroundColor: '#fff',
  alignItems: 'center',
  justifyContent: 'center',
},
});
```



Atividade Prática 2

- Como eu já tenho o título do menu inferior, eu posso omitir o título geral da tela com o uso da prop **headerShown**.

```
const Stack = createNativeStackNavigator();  
return (  
  <NavigationContainer>  
    <Stack.Navigator>  
      <Stack.Screen name="Despesas" component={BottomTabScreen}  
        options={{headerShown:false}}/>  
      <Stack.Screen name="GerenciarDespesa" component={GerenciarDespesa} />  
    </Stack.Navigator>  
  </NavigationContainer>  
);
```



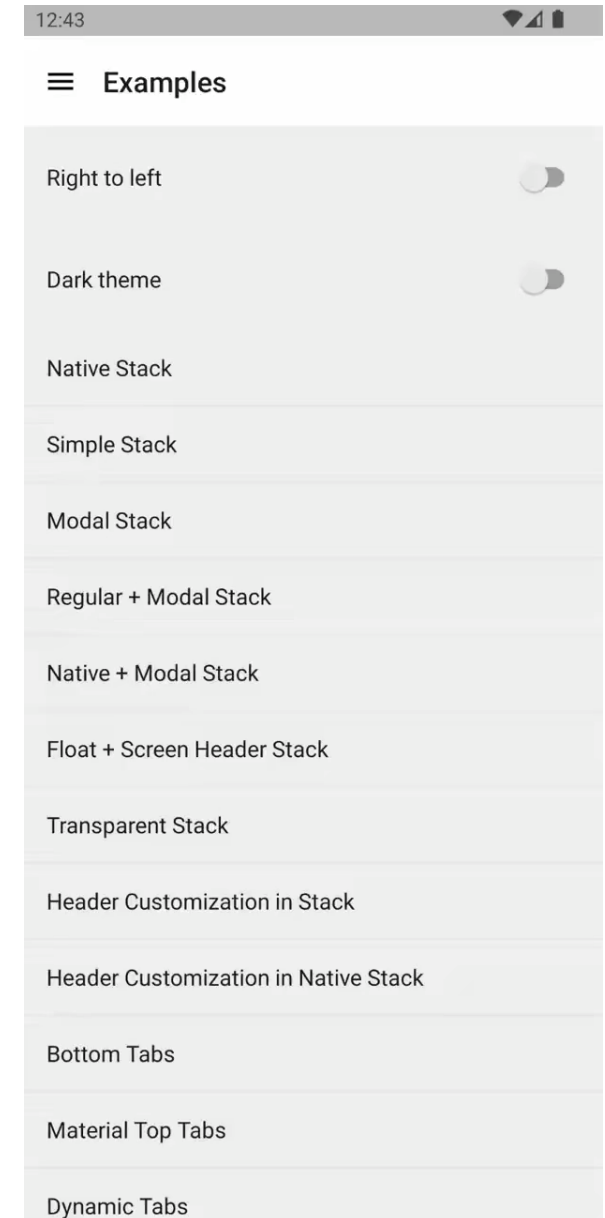


Atividade **Prática** 2

- O que fizemos...
 - DespesasRecentes e TodasDespesas ficaram no **Tab** e são agrupadas em uma tela chamada BottonTabScreen.
 - GerenciarDespesas e a tela agrupada BottonTabScreen ficaram na **Stack**.

Drawer

- Menu lateral (tipo “hambúrguer”) — desliza da esquerda (ou direita).
- Bom para apps com muitas seções.
- Menu retrátil que mostra opções de tela.





Drawer

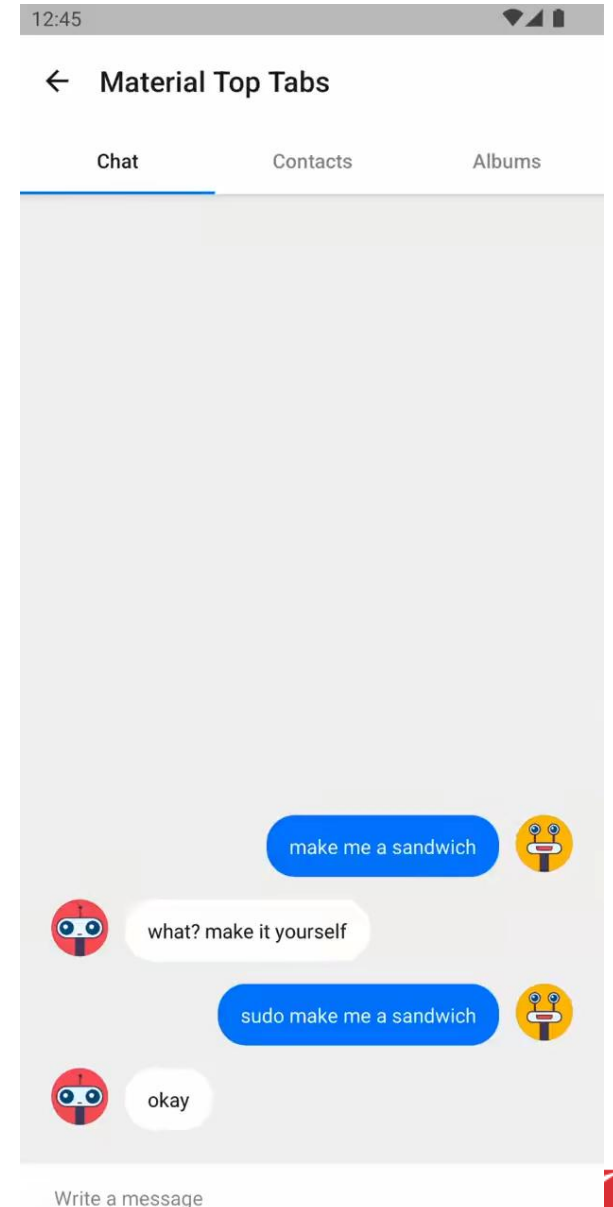
- Para usar este navegador, certifique-se de ter instalado:
 - `npm install @react-navigation/Native`
 - `npx expo install react-native-gesture-handler`
 - `npm install @react-navigation/drawer`
 - `npx expo install react-native-reanimated`

```
import { createDrawerNavigator } from '@react-navigation/drawer';  
const Drawer = createDrawerNavigator();
```

```
function MyDrawer() {  
  return (  
    <Drawer.Navigator>  
      <Drawer.Screen name="Home" component={HomeScreen} />  
      <Drawer.Screen name="Profile" component={ProfileScreen} />  
    </Drawer.Navigator>  
  );  
};
```

Material Top Tabs

- Uma barra de abas com tema de Material Design na parte superior da tela.
- Permite alternar entre diferentes rotas tocando nas abas ou deslizando horizontalmente.
- As transições são animadas por padrão.
- Os componentes da tela para cada rota são montados imediatamente.



Material Top Tabs

- Para usar este navegador, certifique-se de ter instalado:
 - `npm install @react-navigation/native`
 - `npm install @react-navigation/material-top-tabs`
 - `npx expo install react-native-pager-view`

```
import { createMaterialTopTabNavigator } from '@react-navigation/material-top-tabs';

const Tab = createMaterialTopTabNavigator();

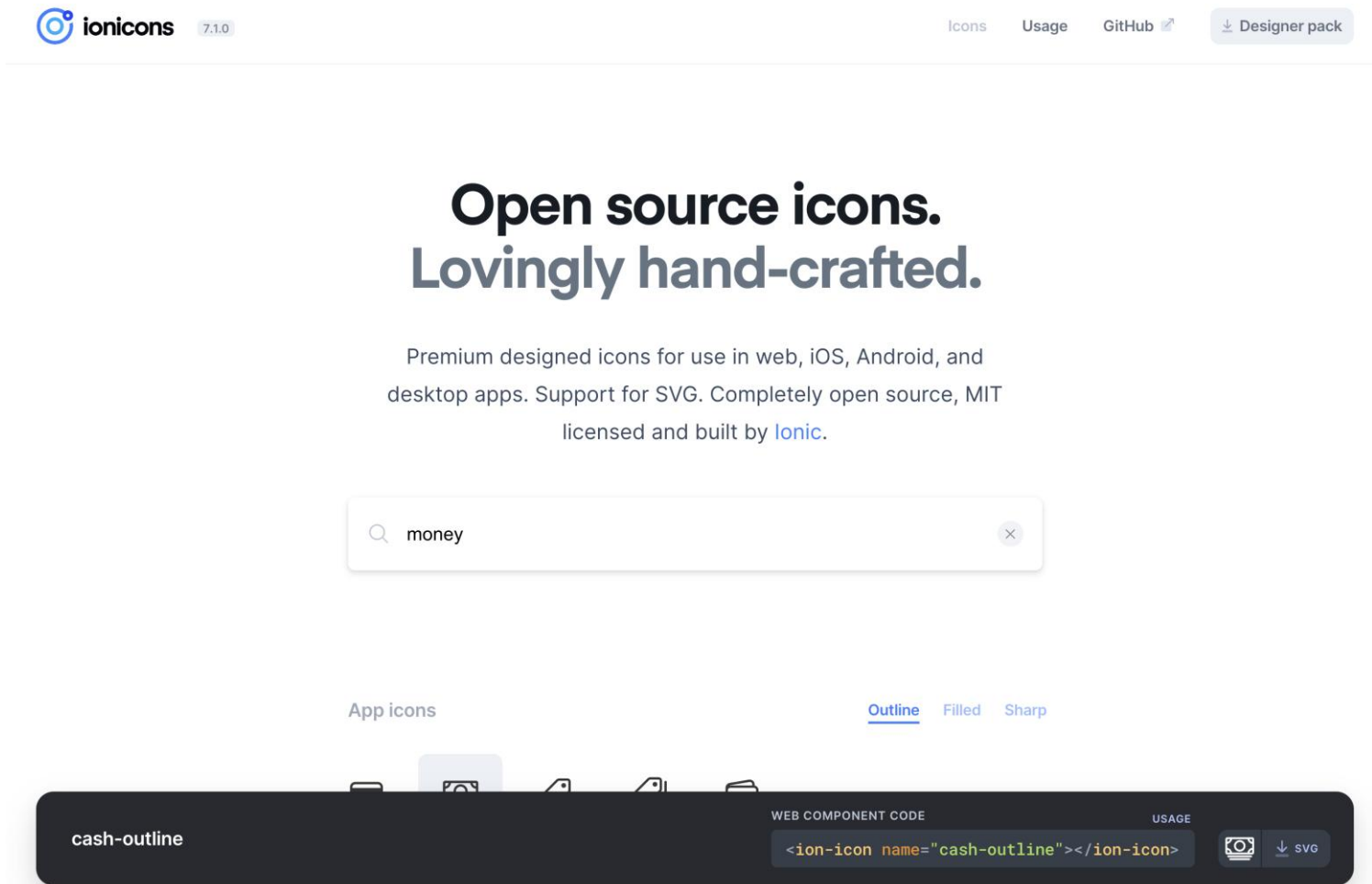
function MyTabs() {
  return (
    <Tab.Navigator>
    <Tab.Screen name="Home" component={HomeScreen} />
    <Tab.Screen name="Profile" component={ProfileScreen} />
    </Tab.Navigator>
  );
}
```



Ícones

- No React Native, ícones são componentes que **representam imagens vetoriais pequenas** (tipo uma casinha para “Home”, uma engrenagem para “Settings” etc).
- Eles são muito usados para:
 - Botões
 - Menus (Drawer, Bottom Tabs)
 - Cabeçalhos (Headers)
 - Feedback visual (check, warning, etc.)

- É uma biblioteca de ícones criada pela equipe do **Ionic Framework**.

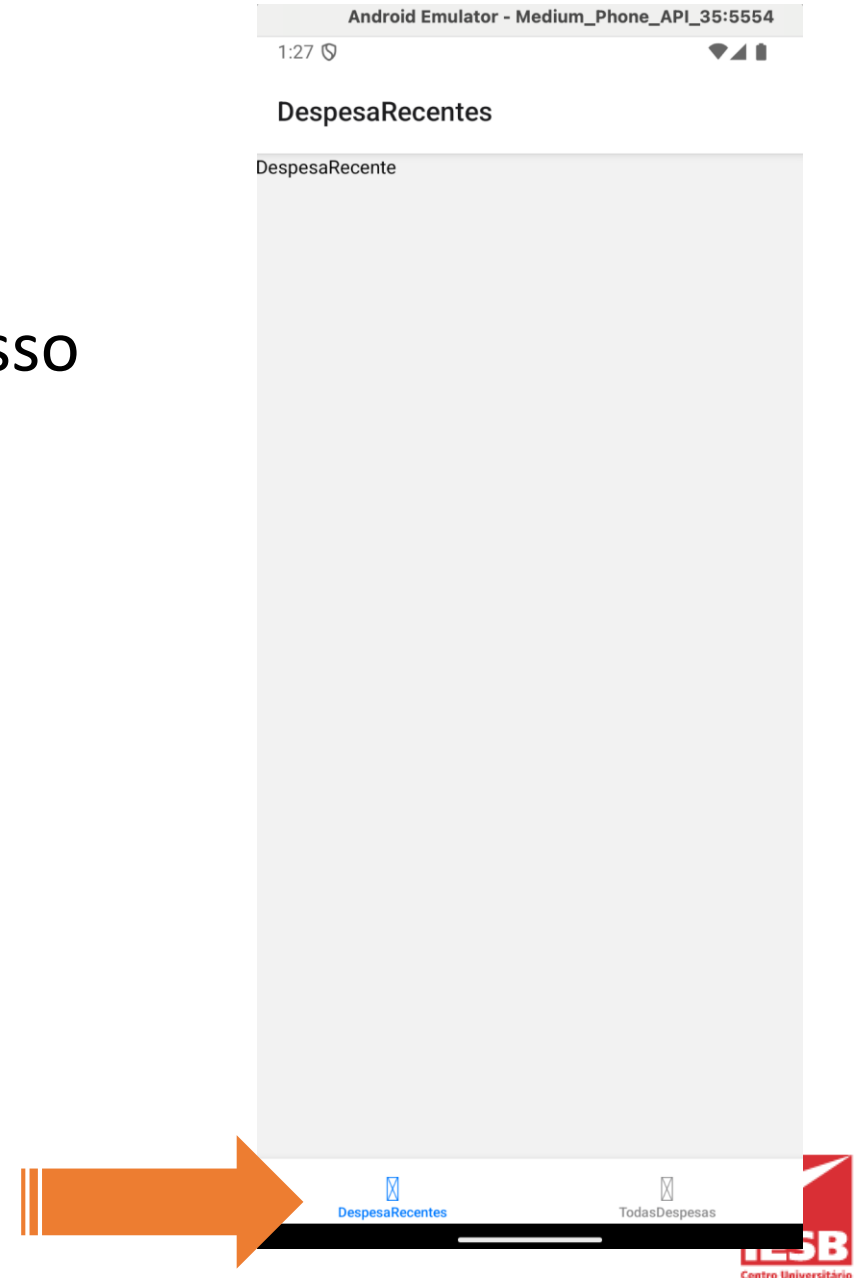


<https://ionic.io/ionicons>

Atividade **Prática** 2

- Vamos adicionar ícones no menu inferior no nosso aplicativo.
- Para isso, precisamos importar:

```
import { Ionicons } from '@expo/vector-icons';
```



Atividade Prática 2

```
import { Ionicons } from '@expo/vector-icons';
```

```
function BottonTabScreen(){
```

```
  return (
```

```
    <Tab.Navigator>
```

```
      <Tab.Screen name="DespesasRecientes" component={DespesasRecientes}
```

```
      options={{tabBarIcon: ({color, size}) => (<Ionicons name="hourglass" size={size} color={color} />),
```

```
      tabBarLabel: 'Recientes',
```

```
      title: 'Despesas Recientes',
```

```
      tabBarLabelStyle: { fontSize: 12 }}}
```

```
    />
```

```
    <Tab.Screen name="TodasDespesas" component={TodasDespesas} />
```

```
  </Tab.Navigator>
```

```
)
```

```
}
```

title

Despesas Recientes

Despesa Recente

Arrow function

tabBarIcon

tabBarLabel

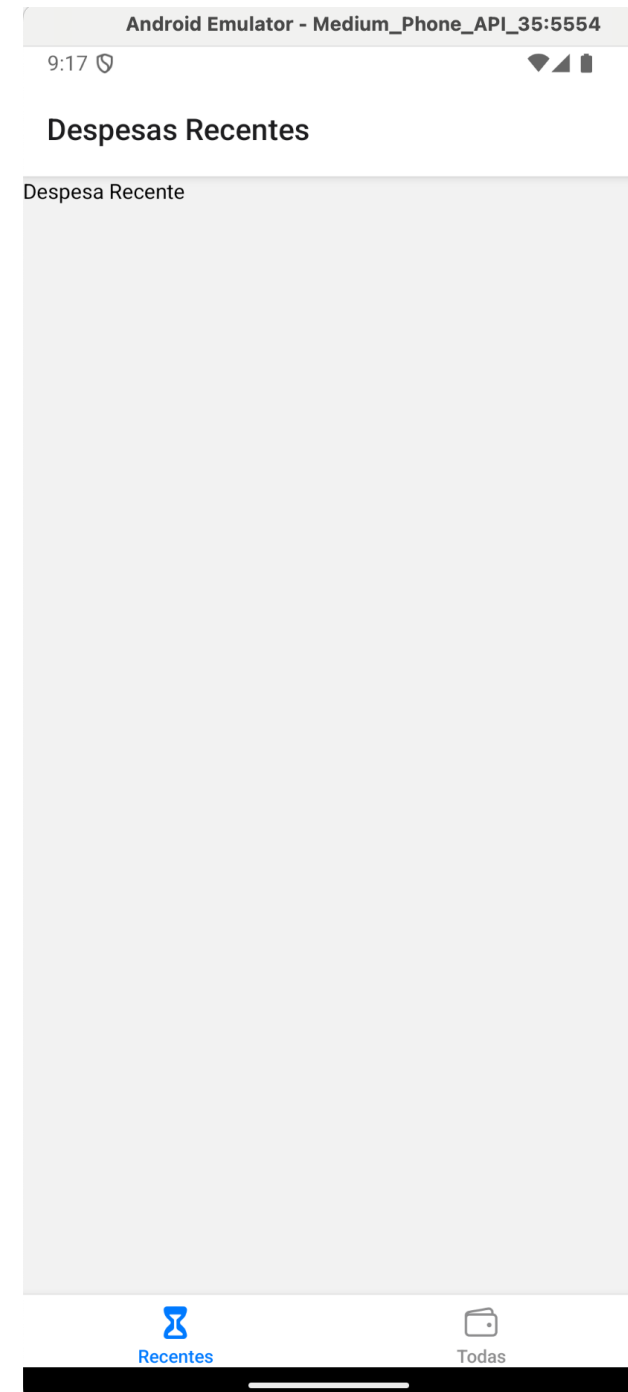
Recientes

TodasDespesas



Atividade

- Adicione um ícone a Aba TodasDespesas: selecione um ícone que tem a ver com o aplicativo.
- Adicione o título – Todas as Despesas
- Adicione o rótulo da Aba – Todas
- Configure o tamanho da fonte do rótulo para 12.





Atividade Prática 2

```
function BottonTabScreen(){
  return (
    <Tab.Navigator >

      <Tab.Screen name="DespesasRecentes" component={DespesasRecentes}
        options={{tabBarIcon: ({color, size}) => (<Ionicons name="hourglass"
          size={size} color={color} />),
          tabBarLabel: 'Recentes',
          title: 'Despesas Recentes',
          tabBarLabelStyle: { fontSize: 12 }}}
        />

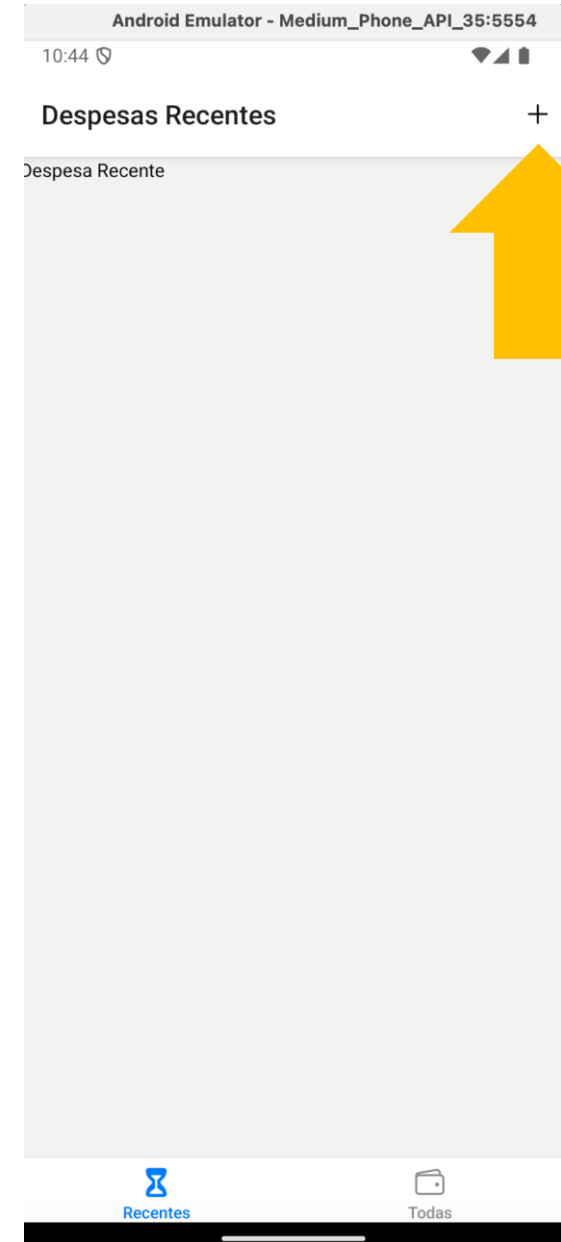
      <Tab.Screen name="TodasDespesas" component={TodasDespesas}
        options={{tabBarIcon: ({color, size}) => (<Ionicons name="wallet-outline"
          size={size} color={color} />),
          tabBarLabel: 'Todas',
          title: 'Todas as Despesas',
          tabBarLabelStyle: { fontSize: 12 }}}
        />

    </Tab.Navigator>
  )
}
```

Componente **IconButton**

- Agora vamos criar um ícone botão para adicionar novas despesas.
- Para isso, crie um componente chamado `IconButton.js`, dentro da pasta `components`, com a seguinte assinatura da função principal:

```
function IconButton({ icon, size, color, onPress }) {}
```



Componente **IconButton**

- Essa sintaxe com { icon, size, color, onPress } nos parâmetros da função é chamada de **desestruturação de props**.

```
1  import {Text} from 'react-native';
2
3
4  function IconButton({ icon, size, color, onPress }) {
5    return (
6      <Text></Text>
7    );
8  }
9
10 export default IconButton;
```



Atividade

- Faça com que a função retorne um componente <Pressable>, com o valor do atributo onPress definido com o valor onPress.
- Dentro de <Pressable>, adicione uma <View> filha.
- Dentro da <View> filha, adicione um componente <Icons>, com as props name, size e color configuradas com os valores da assinatura da função: name=icon, size=size, color=color.

```
function IconButton({ icon, size, color, onPress }) {}
```

Atividade Prática 2

```
1  import {Pressable, View} from 'react-native';
2  import { Ionicons } from '@expo/vector-icons';
3
4
5  function IconButton({ icon, size, color, onPress }) {
6    return (
7      <Pressable onPress={onPress} >
8        <View >
9          <Ionicons name={icon} size={size} color={color} />
10        </View>
11      </Pressable>
12    );
13  }
14
15  export default IconButton;
```



Estilos

- Precisamos ajustar os estilos do novo componente para que melhor se ajuste a tela.
- Precisamos ajustar também o estilo para quando o botão seja pressionado (pressed).



Atividade Prática 2

aula_09 > components > JS IconButton.js > ...

```
1  ∨ import {Pressable, StyleSheet, View} from 'react-native';
2  import { Ionicons } from '@expo/vector-icons';
3
4
5  ∨ function IconButton({ icon, size, color, onPress }) {
6  ∨   return (
7  ∨     <Pressable onPress={onPress}
8     style={({pressed}) => pressed && styles.pressed}>
9  ∨       <View style={styles.button_container}>
10        <Ionicons name={icon} size={size} color={color} />
11      </View>
12    </Pressable>
13  );
14  }
15
16  export default IconButton;
17
18  ∨ const styles = StyleSheet.create({
19  ∨   button_container: {
20     padding: 10,
21   },
22  ∨   pressed: {
23     opacity: 0.5,
24   }
25  });
26
27
```

pressed **vem automaticamente do Pressable** e:

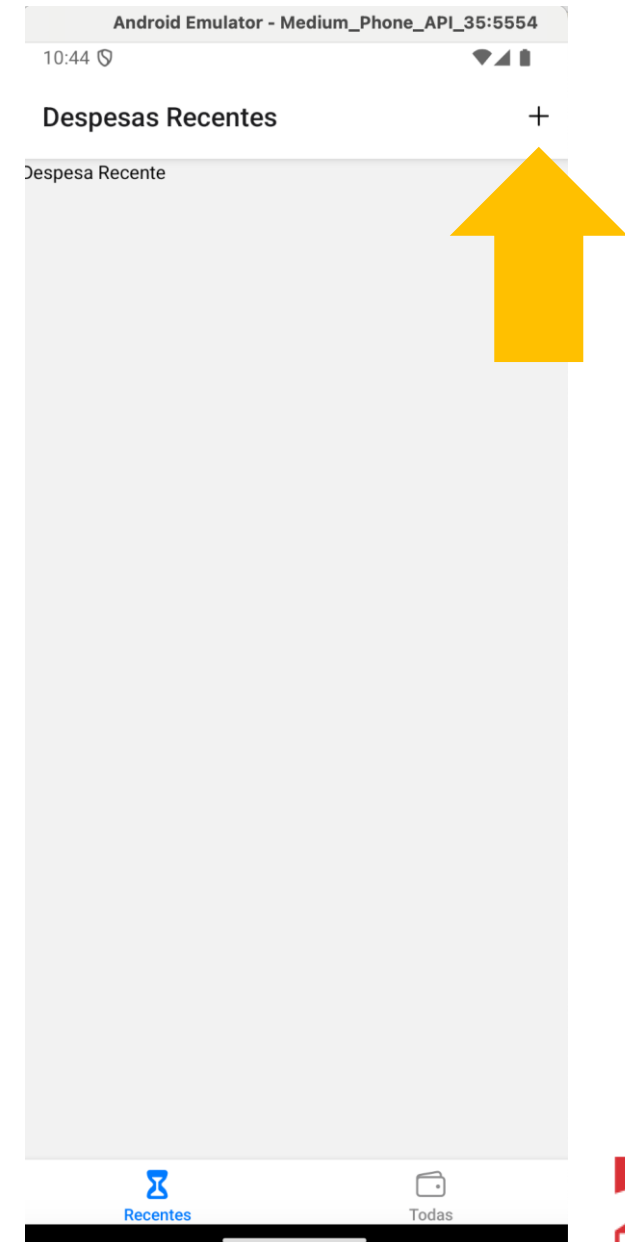
-> é **true** enquanto o usuário está apertando o botão.

-> é **false** quando não está tocando.

Ajuste seu código para utilizar nova definição de estilos no o IconButton!

Botão do Topo

- Agora precisamos adicionar o nosso novo componente no topo, lado direito da tela inicial.

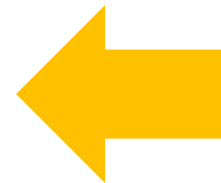


Atividade Prática 2

```
function BottonTabScreen(){
  return (
    <Tab.Navigator
      screenOptions={{ headerRight: () => <IconButton
        icon="add" size={24} onPress={() => {}} /> }}>

      <Tab.Screen name="DespesasRecentes" component={DespesasRecentes}
        options={{tabBarIcon: ({color, size}) => (<Ionicons name="hourglass"
          size={size} color={color} />),
          tabBarLabel: 'Recentes',
          title: 'Despesas Recentes',
          tabBarLabelStyle: { fontSize: 12 }}}
        />
      <Tab.Screen name="TodasDespesas" component={TodasDespesas}
        options={{tabBarIcon: ({color, size}) => (<Ionicons name="wallet-outline"
          size={size} color={color} />),
          tabBarLabel: 'Todas',
          title: 'Todas as Despesas',
          tabBarLabelStyle: { fontSize: 12 }}}
        />
    </Tab.Navigator>
  )
}

return (
```



No arquivo App.js



Adicionar **Navegação**

- Por fim, vamos agora adicionar a **navegação** para a tela de adicionar despesa.
- Para isso, vamos usar o hook **useNavigation**!

```
import {useNavigation} from '@react-navigation/native';
```

useNavigation

- O useNavigation é **um hook** do React Navigation que te dá acesso ao **objeto de navegação (navigation) dentro de componentes que não são telas diretamente** — tipo um botão, um componente customizado, um item da lista etc.
- Serve pra você poder **navegar entre telas (screens), voltar, ou acessar informações da navegação mesmo fora do componente de rota** (ou seja, fora de um <Screen /> do Navigator).



useNavigation

Método	O que faz
<code>navigation.navigate('Rota')</code>	Vai pra uma nova tela
<code>navigation.goBack()</code>	Volta pra tela anterior
<code>navigation.replace('Rota')</code>	Isso troca a tela atual por uma nova e não deixa a anterior no histórico.
<code>navigation.push('Rota')</code>	Empilha novamente a mesma tela. Quando você quer navegar várias vezes pra mesma tela com dados diferentes (tipo uma tela de perfil pra diferentes usuários).



Atividade **Prática** 2

- Vamos adicionar navegação no ícone que adicionamos no canto superior direito do nosso App.

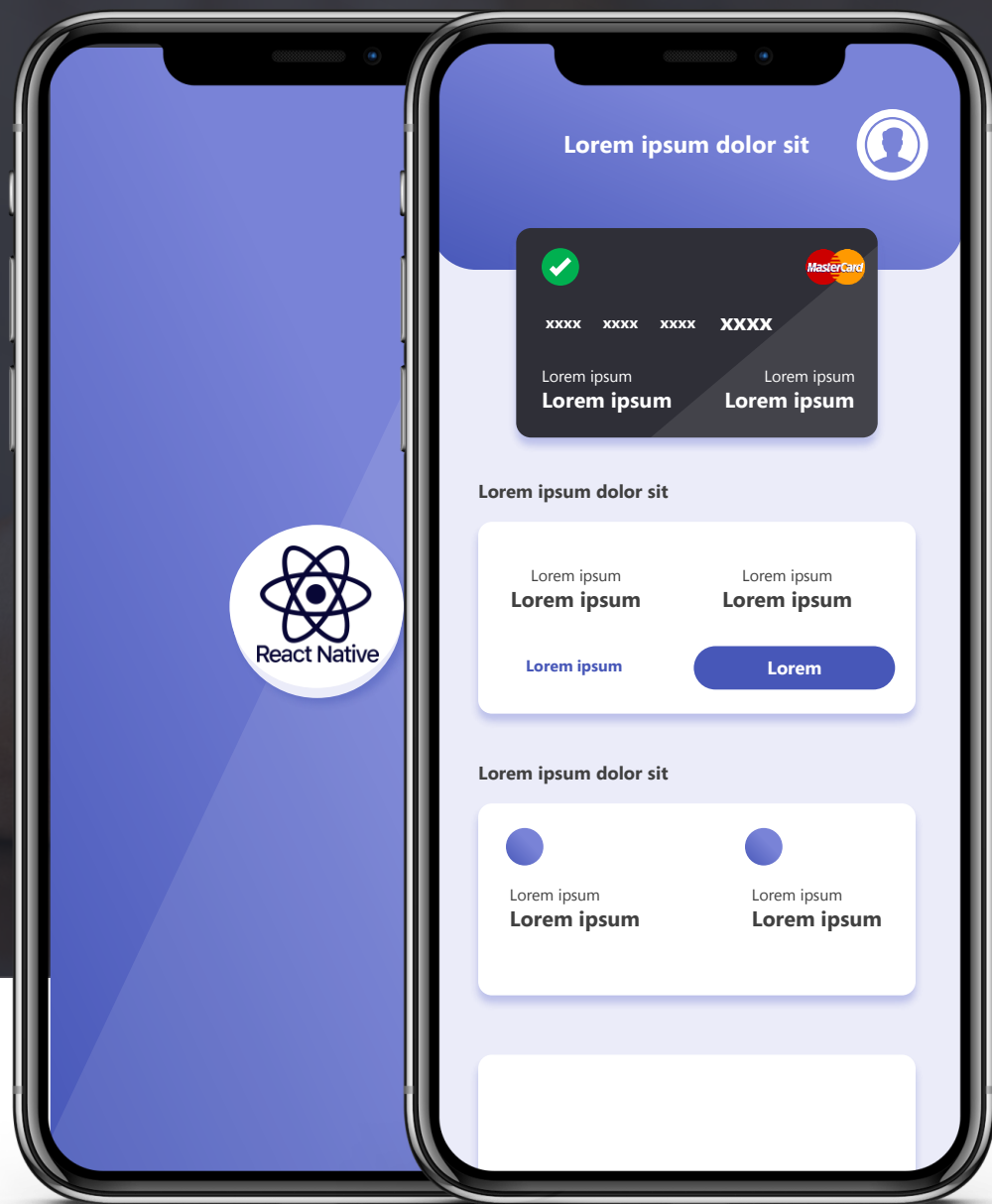


Atividade Prática 2

```
JS App.js ×
aula_09 > JS App.js > App

19
20 function BottonTabScreen(){
21   const navigation = useNavigation();
22
23   return (
24     <Tab.Navigator
25       screenOptions={({ navigation }) => ({ headerRight: () => <IconButton
26         icon="add" size={24} onPress={() => {
27           navigation.navigate('GerenciarDespesa')
28         }} /> }) }>
29
30       <Tab.Screen name="DespesasRecentes" component={DespesasRecentes}
31         options={{tabBarIcon: ({color, size}) => (<Ionicons name="hourglass"
32           size={size} color={color} />),
33         tabBarLabel: 'Recentes',
34         title: 'Despesas Recentes',
35         tabBarLabelStyle: { fontSize: 12 }}}
36       />
37       <Tab.Screen name="TodasDespesas" component={TodasDespesas}
38         options={{tabBarIcon: ({color, size}) => (<Ionicons name="wallet-outline"
39           size={size} color={color} />),
40         tabBarLabel: 'Todas',
41         title: 'Todas as Despesas',
42         tabBarLabelStyle: { fontSize: 12 }}}
43       />
44     </Tab.Navigator>
45   )
46 }
47
```

Veja que foi preciso transformar a entrada em uma arrow function com o parâmetro navigation de entrada



DÚVIDAS